

RESEARCH

Open Access



Zero-trust based dynamic access control for cloud computing

Ri Wang^{1,2}, Chen Li^{1,2*} , Kun Zhang^{1,2} and Bibo Tu^{1,2}

Abstract

Most corporations and organizations rely heavily on access control to protect data accessibility and enable resource sharing across networks and departments. However, with the development of cloud computing, traditional boundary protection struggles to mitigate the increasing attacks and threats. In addition, most existing dynamic access control methods match static rules with dynamic metrics, which cause system damage through their delayed responses to threats and attacks. The zero-trust architecture (ZTA) provides continuous authentication and dynamic authorization for all users to accommodate the security demands of cloud computing. Drawing inspiration from the ZTA, we first present a TBAC (Trust-based Access Control) model and design a trust assessment methodology to update user trustworthiness. Then, we introduce dynamic rules in the TBAC model to implement a dynamic access control system DR-TBAC (TBAC with Dynamic Rule). We apply the DQN (Deep Q-Network) algorithm to dynamically update the trust thresholds based on static rules comparing dynamic trust with predefined trust thresholds to achieve adaptive access control policies. In this paper, we rebuild the cloud security access environment from the perspective of dynamic trust and rule optimization and strengthen the constraints on user behaviors throughout the access control lifecycle of cloud computing. Finally, a thorough analysis and assessment regarding offline training models and the online deployment of the DR-TBAC system into the cloud platform highlight its security and accuracy relative to baseline models.

Keywords Access control, Dynamic rule, Deep reinforcement learning, Dynamic trust, Cloud computing

Introduction

The widely used cloud computing technology stores, processes, and manages data on remote servers centered on the Internet, providing users with a range of resources in the form of services. Although the efficiency of information sharing and the use of computer and storage resources has grown due to the growth of cloud computing, new and growing security risks have also been introduced.

Traditional network environments are built on border defenses and trusted internal networks, where resources and users are relatively fixed, privilege management changes are infrequent, and user access is limited by physical location. The dynamic, scalable, multi-tenant, and rapid availability of services in cloud computing environments breaks down these constraints. On the cloud platform, multiple tenants sharing infrastructure leads to security threats such as data leakage or overstepping of authority. Moreover, users can access cloud resources that automatically increase or decrease based on real-time demand from anywhere, i.e., user behavior in internal networks is no longer completely trustworthy. Once the attackers, such as malevolent cloud administrators, have gained access to the internal networks, they can easily compromise target resources because they hide malicious behavior from detection by the enterprise.

*Correspondence:

Chen Li
lichen@iie.ac.cn

¹ Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100093, China

² School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100049, China

However, traditional IAM systems, such as role-based access control (RBAC), mandatory access control (MAC), discretionary access control (DAC), and attribute-based access control (ABAC)(Kesarwani et al. 2022), are not flexible and scalable enough for permission management to provide a cost-effective solution, which makes threat scenarios such as compromised or lost credentials, insider threats, and incorrectly configured policies more frequent. Therefore, the identity and access management (IAM) system in cloud computing environments must be dynamic, flexible, and instantaneously responsive to handle the swift alterations in cloud resources, multi-tenant complexity, and globalized access needs.

Zero trust (ZT) refers to an evolving cybersecurity concept and model that have emerged in response to the changing nature of cyber threats and network environments. It moves cybersecurity defense from traditional perimeter-based security models to a more dynamic and adaptive security posture, with the premise that implicit trust is never given but needs to be continually assessed(Stafford 2020). As the concept of zero-trust security evolves and gains popularity, the ultimate goal is to protect data across an increasingly fragmented information fabric, which not only provides security across all access points but also maintains a seamless user experience and work efficiency. More specifically, ZT replaces traditional manual analysis and application with responsive and collaborative automated access processes and policy enforcement defenses by requiring continuous discovery, monitoring, and assessment. Compared with the targeted countermeasure of after-the-fact defense against threats and attacks, user management and control from the access origin can essentially achieve traceability and threat mitigation.

ZTA provides a more secure, flexible, and adaptive security IAM for cloud computing environments based on the principle of “always verify, never trust”. Rather than depending on physical or virtual network barriers to protect resources, the ZTA enforces strict authentication and least privilege authorization on all access attempts. It redefines the user certificate authority mechanism by taking the dynamic trust degree as the foundational credential for subject access and resource management, and it ensures that security policies are updated in tandem with environmental changes by evaluating and controlling each access request in real-time. As shown in Fig. 1, the core components of ZTA are the policy decision point (PDP) running in the control panel and the policy enforcement point (PEP) running in the data panel. The PDP assesses access requests and decides whether to grant them based on a set of dynamic policies that require a trust level that takes into account access subject information, the requesting

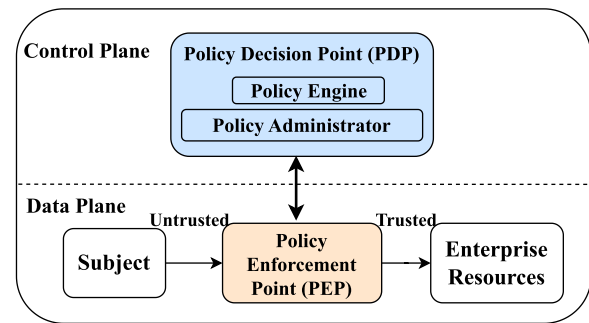


Fig. 1 Core structure of the ZTA

system’s observable status, resource-related attributes, and other behavioral attributes. On the other hand, the PEP is positioned at the system’s access control boundary and carries out the decisions made by the PDP.

The core of designing the IAM system based on ZTA is trust assessment and policy update(García-Teodoro et al. 2022a; Yao et al. 2020; Jin et al. 2021), which requires continuous monitoring and analysis of access subjects and contexts. By utilizing machine learning (ML) technology, the system detects anomalous patterns of behavior, such as irregular login attempts or peculiar data access patterns(Miao et al. 2021), and dynamically adjusts authorization policies during the access control procedure to guarantee security and compliance. In particular, reinforcement learning (RL) can learn, adapt, and make decisions in complex and high-dimensional environments, as well as seek an optimal sequential decision policy without human supervision(Pouyanfar et al. 2018) to optimize long-term objectives and efficiently generalize complex policies. It is essential for secure access control mechanisms that track user interactions to ensure long-term system security and integrity. Deep offline reinforcement learning, which combines the powerful learning capabilities of deep learning with the powerful decision-making capabilities of offline RL, can be used to train powerful agents without environmental interaction by leveraging deep neural networks and massive offline datasets.

To effectively deal with the security challenges posed by the borderless, dynamic, and multi-tenant nature of cloud environments, we propose the TBAC model and introduce dynamic rules based on this model to implement a secure and dynamic access control system architecture. To ensure security and compliance, the core of this architecture is the dynamic modification of access privileges based on updated policy rules for offline RL agents and real-time updates of user trustworthiness. This transition follows the core principles

of ZTA and the security requirements of cloud data centres. The main contributions of this paper are summarized as follows.

- A TBAC model is proposed for authorization based on dynamic trust and trust thresholds. User trustworthiness is quantified by trust assessment methodology based on the LSTM (long short-term memory) model. It effectively fuses user behavioral and contextual information recorded in logs to detect the user trustworthiness in real-time.
- The DR-TBAC system that tracks the whole process of user interactions with the cloud platform in real-time is proposed. The MDP describes the environment changes combined with user trustworthiness as state vectors for the DQN algorithm to make the optimal decision. The DR-TBAC system automatically adjusts access control rules based on user behavior and environment changes, responds to insecure user behaviors in time, and avoids the occurrence of unsafe interactions.
- The open-source OpenStack log dataset is used to train the trust assessment model and to generate experience tuples for training the DQN algorithm after preprocessing to find the optimal hyperparameter selection. To demonstrate the practicality and effectiveness of the DR-TBAC system, we redefine the policy file of the OpenStack Nova component and analyze several representative cases of users accessing the OpenStack cloud platform. Finally, we compare the DR-TBAC system with seven baseline models to demonstrate how accurate and secure it is compared to the alternative approach.

The rest of this article is summarised: “Related Work” describes the current state of research for this work. “Model Design” details the TBAC model and the trust assessment methodology. “System Implementation” details the dynamic access control system implementation. In “Results”, we perform a detailed experimental analysis of the online and offline DR-TBAC system in OpenStack. Finally, we conclude this work and discuss the future work in the “Conclusion”.

Related work

Developing the concept of trust is the foundation for making dynamic access control decisions based on ZTA. Riad et al. (2017); Yao et al. (2020); Mehraj et al. (2021) proposed trust-based access control models that performed trust assessment based on predefined trust-related attributes using subjective methods. This subjectivity can lead to inconsistencies in trust assessments, compromising the fairness and consistency of

access control decisions. Jing et al. (2021); Khan et al. (2023); Zhao et al. (2021) start to take into account factors like user behavior, environmental security, etc., but they quantify these factors into numerical values. This simplification may ignore or lose certain essential contextual information, resulting in an incomplete or incorrect trust assessment. These methods essentially use precise, static approaches to describe and deal with complex, dynamic trust relationships.

To analyze these multidimensional attributes that are difficult to observe firsthand, Afshar et al. (2021); Alshammari et al. (2021) employed ML techniques such as Linear Regression (LR), Random Forest (RF), and KNN to identify malicious users, which treated trust assessment as a binary classification problem. They assigned a subset of resources to each user in advance and PDP decided whether the user accessed the resources. However, trust assessment methods based on real-time data analysis and continuous learning can enhance the security and flexibility of ZTA. A lot of work has been presented to learn user normal behavioral patterns online and using time series models like neural networks (NN) (Li et al. 2022; Zhang 2022). To effectively forecast user behavioral patterns, our suggested trust assessment approach not only learns user behavioral data but also retains significant contextual information at the time of behavior.

Another key part of implementing dynamic access control based on ZTA is the dynamic adjustment of authorization policies. Dimitrakos et al. (2020) conditioned ABAC authorization policies on trust assessment and fixed trust thresholds, and continuously detected changes and re-evaluated authorization rules and trust levels throughout the authorization lifecycle. García-Teodoro et al. (2022b) designed a dPDP module that dynamically estimates a user's security profile based on specific security-related attributes using MSNM (Multivariate Statistical Network Monitoring). The dPDP module compares each security profile with predefined upper control limits (UCLs) to make access decisions. Zhao et al. (2022) designed a zero-trust access control mechanism that mined the user behavior data using the fuzzy clustering algorithm to assess trust level and determine whether the access request is within the permission perimeter. Based on the RBAC model, Yang et al. (2022) introduced a deep learning-based real-time user trust assessment model and different levels of privileges to set different user trust thresholds. The user's trust determines the validity period of the token. The fact that user trustworthiness is re-evaluated after the validity period expires demonstrates the real-time nature of trust evaluation. We discover that the above research all define static and coarse-grained policies, which do not allow for an efficient and adaptive access control system.

For IAM systems to make more accurate authorization decisions, research should not only utilize smarter trust assessment methods but also continuous policy improvement. The RBAC model serves as the foundation for Zhou et al. (2019); Tay et al. (2020); Kim and Cho (2019) to automatically update user roles, i.e., automatic update of user privileges, by detecting user behavior or learning contextual features through various ML techniques. Srivastava et al. (2021) employed an autoencoder to identify user behavioral patterns and calculate user trustworthiness by taking into account the time and location of the request, the requester's history behaviors, and the sensitivity of the requested information. The Random Forest decision model produces a binary decision of whether or not to authorize based on the trust value, but it is unable to learn and dynamically adjust the policy in response to environmental feedback. Jin et al. (2021) constructed dynamically updated user trust profiles with a bidirectional GRU using user daily behavior logs and learned optimal traffic scheduling policies based on the Multi-Agent Deep Deterministic Policy Gradient (MADDPG) for adaptive and automatic collaborative management. However, this article addressed the dynamic access control problem from the perspective of dynamic resource scheduling based on network security. Similarly, Mandal et al. (2021) implemented an access control policy based on zero-trust networks from the perspective of network traffic, which assigns thresholds to port numbers and dynamically updates the thresholds using the additive-increase and multiplicative-decrease (AIMD) algorithm. We find that current research on implementing dynamic access control based on ZTA focuses on dynamic trust evaluation and pays less attention to policy rule optimization. Table 1 compares our proposed DR-TBAC to some of the most relevant. The DR-TBAC system is capable of assessing user trustworthiness in real-time by integrating multi-dimensional factors, and automatically updating access control policies to achieve truly dynamic authorization by perceiving both user behavior and environmental changes.

Model design

Formal description Of The TBAC model

Different access control models provide a clear permission distinction for cloud users. Each user on the cloud platform obtains permissions based on specific conditions, i.e., performs operations on specific objects. The static decision mechanism can prevent potentially malicious users from interacting with the object but can lead to harmful security breaches if malicious users completely replace legitimate users to access cloud resources. Therefore, policy decisions are not absolute and need to be adjusted based on user behavioral performance throughout the time horizon. A popular approach to dynamic authorization is to compare trust thresholds that have been pre-established with real-time computed trust scores. In this section, we formally describe a minimum collection of the elements and assignment relationship of the TBAC model:

- 1 Users: People who use the cloud resources or services are denoted as U , where the element is denoted as u .
- 2 Object: Cloud resources or services are denoted as O , where the element is denoted as o .
- 3 Permission: The qualification to perform operation on object o is denoted as P , where the element is denoted as p .
- 4 Trust: User trust calculated based on the trust assessment methodology, denoted as T .
- 5 Trust Threshold: The minimum level of trust required to access object. The threshold is dynamically adjusted according to trust and changes in the environment, denoted as μ_{tt} .
- 6 Assignment Relationships: Many-to-many user permission assignment set $UP \subseteq U \times P$. For $\forall u \in U, f_{up}(u) = p$ and $p \subseteq P$
- 7 Authorization: If user u is allowed to perform an operation on the object o , then P_A must contain permission to access that object, and the permission trust threshold for that object is less than or equal to current user trustworthiness T , which can be formally described as:
if $P_A = f_{up}(u)$ and $T \geq \mu_{tt}$

Table 1 Comparison of related work

Work	Li et al. (2022)	García-Teodoro et al. (2022b)	Srivastava et al. (2021)	Jin et al. (2021)	DR-TBAC
Behavioral pattern	✓	✓	✓	✓	✓
Contextual information	✗	✗	✓	✓	✓
Dynamic authorization	✓	✓	✓	✓	✓
Adaptive policy	✗	✗	✗	✗	✓

then (u is allowed to access o)

else (u is denied)

Based on the above definition, the TBAC model is shown in Fig. 2.

Trust assessment

As evidence of users' historical security level, constructing accurate user trustworthiness is the core technology in the TBAC model. To make a convincing access authority measure, we design an online trust continuous assessment methodology to detect abnormal user behavior.

We utilize an events-based approach to obtain user behaviors and extract event occurrence commonalities from arbitrary log formats. To handle log data in various forms, we eliminate any unnecessary characters from the original log data. The remaining log data is divided into token fields and stored in a vocabulary V while maintaining integrity. Taking into account the correlation that exists before and after the log fields, we create a sequence of word fragments G and use the n -gram method to intercept word fragments of length n from both directions of the log sequentially. The frequency of each token occurrence in G is recorded in V . To convert sequences of user behavior values into a fixed-size feature matrix, we map token frequency into low-dimensional embedding vectors in a continuous vector space that captures semantic and syntactic relationships between words using fully connected neural networks. As shown in Fig. 3, we select as input the context token that is most relevant to the target token based on the size of n . The word embedding vectors for each token are organized in the order in which they appear in the original log line.

Furthermore, metadata information in logs is critical for analyzing scenarios involving user behavior, such as time and device. Taking the characterization process of time information as an example, legitimate users will follow a day/night cycle-based activity pattern. If we consider both a daily behavioral pattern with an interval

length of 3 h and a weekly behavioral pattern with an interval length of one day, then the temporal information is represented as (d, h) , where $d \in \{0, 1, 2, 3, 4, 5, 6\}$ denotes the day of the week and $h \in \{0, 1, 2, 3\}$ denotes the sequence number of the time slice of a day that is divided into 3-hour intervals. These features are scaled and spliced into vectors of the same dimension as the embedding vectors, representing the 2D matrix features of a single log row.

The trust assessment methodology in Fig. 3 is composed of context LSTM (CLSTM) and bidirectional LSTM (Bi-LSTM). The CLSTM model takes contextual feature vectors and the hidden states $\hat{h}_i$ as input and outputs the hidden states h^i that supply contextual information between log lines over time sequences to the Bi-LSTM model at the next time step. The Bi-LSTM model is bi-directionally trained by input data consisting of hidden states $h^{(i)}$ and behavioral vectors. For every token, the input of the CLSTM model at the next time step is the mean of the hidden states h_i of the forward training output and the hidden states h_i^b of the reverse training output. The probability distribution $p_t = \text{softmax}(h_{t-1}W_p + h_{t+1}^bW_p^b + b_p)$ that each token outputs at moment t through the softmax layer is the probability distribution denoted as predicting the future action based on the current user behavior. The model training target is to find the most similar predicted behavior to the actual behavior.

Given the feature-processed vectors, the model outputs the predicted probabilities of all possible actions and calculates the cross-entropy loss $\text{score}(x, p) = \frac{1}{k} \sum_{t=1}^k H(x_t, p_t)$ between the actual and predicted behaviors as the anomaly score for each log (i.e., each user behavior). In the training procedure, we use stochastic mini-batch and untruncated back-propagation through time to minimize the cross-entropy loss. As a result, abnormal user behaviors receive high cross-entropy loss values during the model launching phase. Finally, we distinguish user events by username and chronological order and quantify the cross-entropy loss obtained by a user u acting as each time point t as user trustworthiness T to represent the degree of deviation in user behavior.

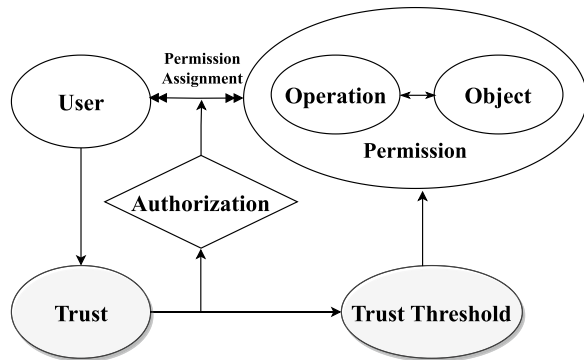


Fig. 2 TBAC model structure

System implementation

System architecture

Trust thresholds defined based on experience knowledge are static, coarse-grained, and do not satisfy the least privilege principle of zero trust. We introduce dynamic rules based on the TBAC model, i.e., train the RL agent offline based on the DQN algorithm to dynamically update the trust thresholds, refine the rules, and

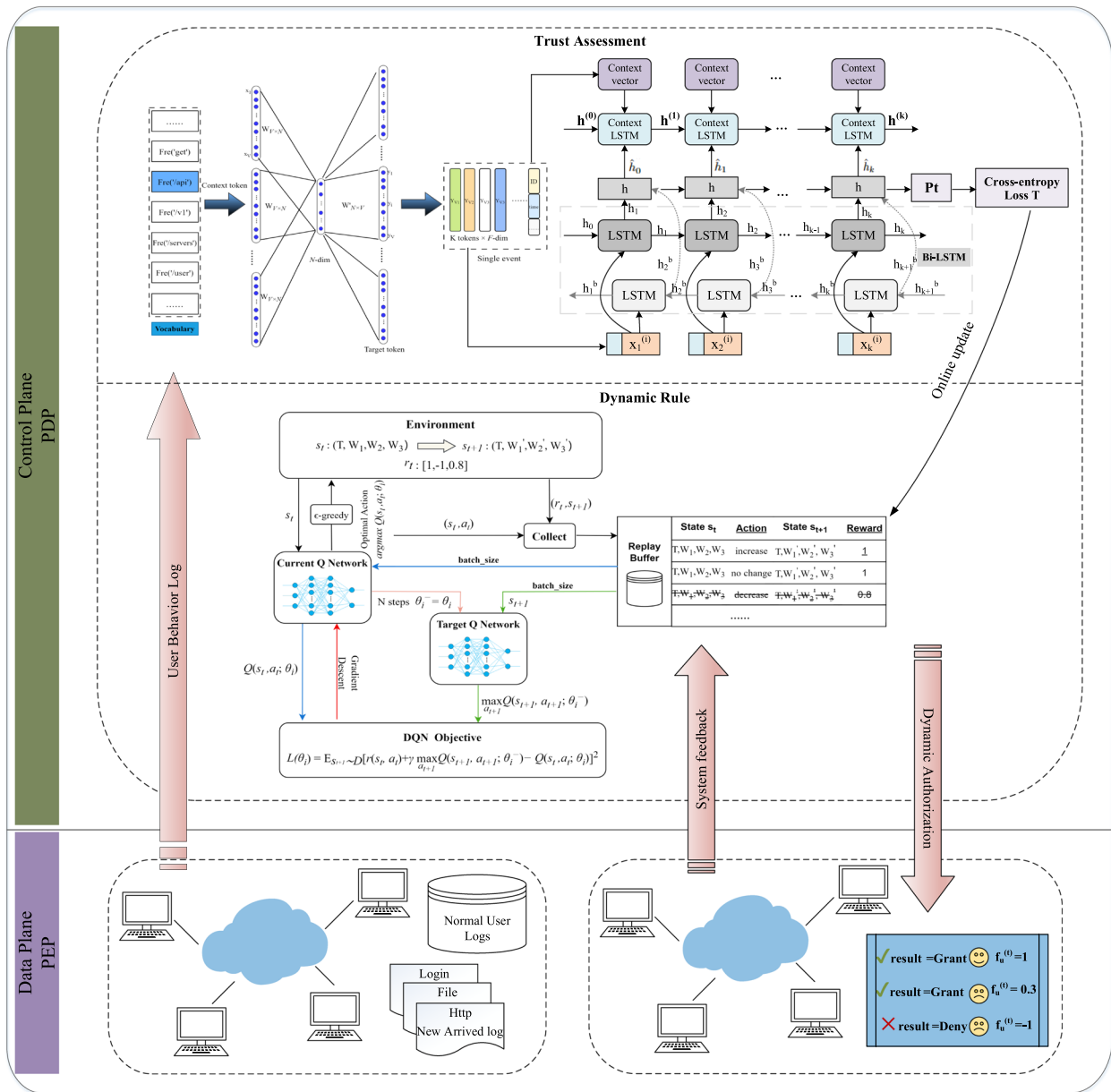


Fig. 3 DR-TBAC system architecture

optimize rule selection to improve the security accuracy for dynamic access to cloud resources.

In conjunction with the ZTA, we implement the PDP component as an adaptive access control system without any human intervention that integrates online anomaly detection to continuously quantify the user trustworthiness and dynamic policy rule updating by offline deep-RL algorithm. As shown in Fig. 3, the user behavior logs are preprocessed to generate behavioral and contextual features and fed into the LSTM model-based trust

assessment method to generate user trustworthiness in real-time. The cloud environment as well as the dynamic user trustworthiness real-time update of the replay buffer serves as experience data for the RL agent to determine the optimal action, i.e., updating trust thresholds. After the DR-TBAC system is launched online, user behavior logs are continuously updated based on user requests, and the trust assessment method continuously updates user trust for the RL agent status updates constantly. The trained RL agent maximizes the long-term system rewards to dynamically update rules that are continually

compared with the dynamic trust to determine the authorization results. Our DR-TBAC system minimizes the risk of after-the-fact revocation of privileges in cloud computing. We next detail the implementation of dynamic rules from RL theory.

RL theory

RL offers a mathematical framework for implementing intelligent interactive dynamic control systems (Cao et al. 2018), and the RL agent is fault-tolerant and tunable to support dynamic updating of policy rules. We record all of the parameters used in Table 2. At every time step t , the RL agent selects an action a_t by observing the current state s_t of the environment and executes the chosen action to interact with the environment. Following this action, the environment transitions to a new state s_{t+1} and outputs a reward $r(s_t, a_t)$ to indicate the immediate desirability of the state-action pair. By receiving these rewards, the agent updates its internal policy or value function to make better decisions and maximize its cumulative rewards. The interaction loop process is the foundation of the RL paradigm. The markov decision process (MDP) framework is commonly used to formalize and analyze the RL problem due to the agent's inability to fully access state information. MDP is characterized by the state space $\mathbb{S}$, the action space $\mathbb{A}$, the reward function $r(s_t, a_t)$, the conditional probability distribution $p(s_{t+1}, r(s_t, a_t) | s_t, a_t)$ and the discount factor $\gamma \in [0, 1]$ (Sutton and Barto 2018), where the conditional probability distribution describes the likelihood of transitioning from one state to another state and captures the dynamics of the environment.

The behavior of the environment over a single time step describes the transition from one state to the next, which

can be denoted as $(s, a) \rightarrow (r, s')$ or typically represented as the expected sum of discounted rewards using the value function:

$$J(\pi) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t \times r(s_t, a_t) \right], \quad (1)$$

where $s_t \sim p(\times | s_{t-1}, a_{t-1})$ represents the transition dynamics, indicating the probability distribution for predicting the next state from the current state and action, and $a_t \sim \pi(\times | s_t)$ represents the action distribution under the policy π at time t . In terms of the entire space of policies π , RL's process of exploring the optimal policy π^* can be expressed as:

$$\pi^* = \operatorname{argmax}_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t \times r(s_t, a_t) \right]. \quad (2)$$

Offline RL

Offline RL is more appropriate for practical applications than online RL. The online RL agent requires iterative interactions with the environment to explore improvement in the policy based on the experience collected, which is equivalent to training the policy using data only from a specific point in time to maximize its long-term reward. However, it is impractical and costly to allow the untrained RL agent to frequently interact with real network environments online due to the highly unsafe nature of the data collection process. Even if some research has been done to deal with the main issues with online RL, such as (Liu et al. 2021) taking advantage of hierarchical deep-RL algorithm with automatic recognition of sub-goal via computer vision (HADS) to overcome the sparse reward problem and boost exploration efficiency, the practical applications of this approach remain restricted. Thus, the advantages of using offline RL are paramount due to the shortcomings and challenges of online RL in practical applications. Offline RL become a hot research topic that multiple off-policy methods to RL can learn near-optimal policies π^* for the desired metrics to maximize long-term rewards from a dataset of behavioral policy π collected offline (Levine et al. 2020).

To optimize the offline RL objective, the action-value function (Q-function) selects a specific action among all possible actions and makes decisions based on the potential long-term rewards associated with different actions in different states. It can be formulated as follows:

$$Q_{\pi}(s_t, a_t) = \mathbb{E}_{\substack{s_t \sim p(s_t, r(s_{t-1}, a_{t-1}) | s_{t-1}, a_{t-1}) \\ a_t \sim \pi(a_t | s_t)}} \left[\sum_{t=0}^{\infty} \gamma^t \times r(s_t, a_t) \right]. \quad (3)$$

Table 2 Summary of key parameters

Parameter	Description
t	Time instance
s	State of the environment
a	Action of the RL agent
r	Reward function
p	Transition dynamics of the environment
γ	Discount factor of cumulative rewards
π/π^*	Current policy/Optimal policy
$J(\pi)$	The expected sum of discounted rewards
$Q_{\pi}(s_t, a_t)/Q^*(s_t, a_t)$	Q-function/Optimal Q-function
θ_i/θ_i^-	DQN parameters/Target DQN parameters
τ	Update Frequency of target network
$L(\theta_i)$	Error objective
y_i	DQN learning target values

The maximized Q-function is expressed as an optimization policy function: $\pi(s_t, a_t) = \operatorname{argmax}_{a_t \in \mathbb{A}} Q(s_t, a_t)$. We derive the Bellman equation that decomposes the Q-function into an immediate reward and a value function for weighted subsequent states $Q_\pi(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p(s_{t+1}, r(s_t, a_t)|s_t, a_t)} [Q_\pi(s_{t+1}, a_{t+1})]$
 $a_{t+1} \sim \pi(a_{t+1}|s_{t+1})$

based on overlapping subproblem property. The model-based dynamic programming algorithm recursively explores the Bellman optimality equation, i.e., the maximum Q-function $Q_{\pi^*}(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p(s_{t+1}, r(s_t, a_t)|s_t, a_t)} [\max_{a_{t+1}} Q(s_{t+1}, a_{t+1})]$. Therefore, the $Q_{\pi^*}(s_t, a_t)$ under the optimal policy π^* is greater than or equal to the $Q_\pi(s_t, a_t)$ under any other policy π for all states and actions. In other words, for every state, the optimal policy π^* should specify the action that leads to the highest expected cumulative reward. In practical circumstances where traversing all states and actions is not practicable, Q-learning (Clifton et al. 2020) can learn to train Q-function based on a limited number of samples and is executed in two main phases: a) based on the known optimal action-value function Q^* , the approximate estimates $Q(s_t, a_t; \theta)$ are continuously improved, which is denoted as $Q(s_t, a_t; \theta) \approx Q^*(s_t, a_t)$, by taking gradient descent on the parameters θ , and b) concerning the current learned policy, the RL agent interacts with a stochastic policy to explore the environment.

In the case of high-dimensional complexity of continuous states and discrete actions, we use the DQN algorithm (Mnih et al. 2015) that designs neural networks instead of the table look-up process. DQN employs a replay buffer that is a data structure to store the RL agent's past experiences that are recorded as tuples $(s_t, a_t, r_{t+1}, s_{t+1})$. The replay buffer contains the agent's state, action, reward, and the subsequent state resulting from its interactions with the environment. The DQN algorithm alternates between gradient descent and data collection to minimize the mean-squared temporal difference (TD) error objective:

$$L(\theta_i) = \mathbb{E}_{s_t, a_t \sim D} [(y - Q(s_t, a_t; \theta_i))^2], \quad (4)$$

which is a measure of the disparity between the predicted Q-values and the target Q-values. The target network

$$y_i = \mathbb{E}_{s_{t+1} \sim D} [r(s_t, a_t) + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}; \theta_i^-)], \quad (5)$$

is estimated by fixed parameters θ_i^- . To improve the overall stability and convergence of the Q-value updating process, the target network parameters (θ_i^-) are updated less frequently than the Q-network's parameters (θ_i). The soft update is accomplished by using a fraction τ of the Q-network parameters and $(1 - \tau)$ of the target network parameters. The DQN algorithm approximates

the optimal policy π^* by minimizing the $L(\theta_i)$, taking gradient descent to continuously tune parameter θ_i . Furthermore, DQN is an off-policy algorithm that computes y_i without the need for the exact knowledge of the initial behavioral policy but learning from the experiences of an exploratory policy, which approximates the optimal policy π^* by using experiences stored in the replay buffer.

Dynamic rule

The improvement process of trust thresholds follows MDP to enhance its interpretability. We describe the state space $\mathbb{S}$ as categorical feature vectors $s_t = [T, W_1, W_2, W_3, W_4]$, which includes the frequency of security events W_3 , request error rate w_1 , access resource sensitivity W_2 , and real-time updated user trustworthiness T . The action a_t is selected based on the current state and reward feedback to determine if trust thresholds have changed, which in turn determines whether a user permission transitions in the next state s_{t+1} . The reward function is crucial in shaping the behavior of the agent. It is defined as a weighted sum of the user's latest trust value after one interaction with the system based on the selected action a_t and the system's feedback value F , denoted as $r(s_t, a_t) = w_t * T + w_f * F$, where $w_t, w_f \in [0, 1]$. When users behave normally, the system gives positive feedback that is determined based on the results of the access execution. The negative feedback indicates that the system denies users' login and authorization or that the authorized user behaves abnormally.

All of the above preparations are intended to load the replay buffer with continuously updated empirical data to recurrently train the RL agent. In Fig. 3, DQN chooses random actions depending on the current state with ϵ probability and adjusts them based on the threshold step τ , which includes threshold increase ($\tau+$), drop ($\tau-$), and no change. At this point, the environment changes, and the reward value is transmitted back to the RL agent, causing the Q-value to be updated. This procedure generates new experience data, which is then collected into the replay buffer. During training, the DQN selects batch-size samples from the buffer, the current Q network computes the current Q value estimate, and the target Q network uses the sample's next state to compute the maximum Q value, copying θ_i to the target network's θ_i^- every N steps. The ultimate TD error is determined for gradient descent to update the parameters θ_i of the current Q network and select the optimal action supporting user permission changes. When the RL agent that has been trained extensively offline deploys in a real environment, it may still need to explore and interact with the environment to fine-tune its policy and adapt to the specific conditions it

encounters. The DR-TBAC system is built on the trained RL agent and dynamically updates rules based on user trustworthiness and environment changes to adjust privilege assignments.

Access control process

Figure 4 presents the workflow of the DR-TBAC system. It is integrated into the cloud as a security module that responds by tracking the log information of cloud services. The dynamic permissions management process of the DR-TBAC system can be described in detail as follows:

- 1) User authentication is carried out by the ID mechanism in the cloud environment when the user logs in. If the authentication is successful, the user obtains the request token and enters the cloud platform to query the historical trustworthiness. Otherwise, the user is denied login. If the user accesses the cloud platform for the first time, the system initializes trust value $T = 0.1$, meaning that the user can perform basic security operations. The user who has successfully logged in initiates a session with the returned token. When the user initiates requests to access services, the DR-TBAC system must determine whether the user has the corresponding permission and whether the user's current trust level meets the trust threshold for activating that permission.
- 2) The log information generated by users interacting with the system provides support for the trust assessment and rule update. The RL agent uses the state vector s_t to select the action a_t with the maximum Q-value, i.e., maintain or adjust the trust thresholds based on the derived optimal policy π^* . If the user has been behaving well, the trust value increases, and the next request can activate higher permission. If the user shows malicious behavior, the trust level decreases. When the trust level decreases below the trust threshold, the permission is revoked, and the

user only activates lower permission for the next request. To make a timely decision on whether to revoke or maintain user permission, the DR-TBAC system must go through the authorization process following each rule update. The system provides feedback values depending on user access results, and the RL agent calculates reward values according to the user trust values and feedback values to determine the next action decision. Thus, the user who obtains an authentication token is not guaranteed to fully execute the corresponding permissions throughout the access process.

The most recent and accurate state information ensures the output of optimal policy rules without affecting the user experience, thus simultaneously ensuring user visibility and privacy from login to the end of the access as well as system security.

Results

The evaluation experiments of the proposed DR-TBAC system are conducted on the server (2.0GHz Intel Xeon v2). The operating system is CentOS Linux 8 with 32GB memory and GTX 1080Ti GPU, and Pytorch 1.9.0 and Python3.6 are used to train models. “Offline evaluation” section determines the hyperparameters of the LSTM and DQN. “Online evaluation” section illustrates the performance enhancement of the DR-TBAC system with the introduction of dynamic rules based on user cases in real-world scenarios by deploying OpenStack (version Victoria) with one control node and one compute node on the server. In addition, the DR-TBAC system demonstrates optimal security accuracy during online deployment compared to other baseline approaches.

Experiment setup

OpenStack, as an open-source cloud computing management platform, provides strong support for evaluating the DR-TBAC system. As shown in Fig. 5, Du et al. (2017) provided OpenStack open source dataset with over one million log entries including VM creation/deletion, stop/start, suspend/resume, and pause/unpause. Among them are 7% abnormal events: 1) neutron timeout during VM creation; 2) libvirt error while destroying a VM; 3) libvirt error during cleanup after destroying a VM. Readable OpenStack logs are a reliable source for analyzing user behavior.

To train the DQN algorithm, we process OpenStack open-source dataset using the ELK Stack tool to generate experience tuples (shown in Fig. 6). Then, to implement the DR-TBAC system to OpenStack, we first define the trust threshold for each permission in OpenStack. OpenStack components have corresponding *policy.json*

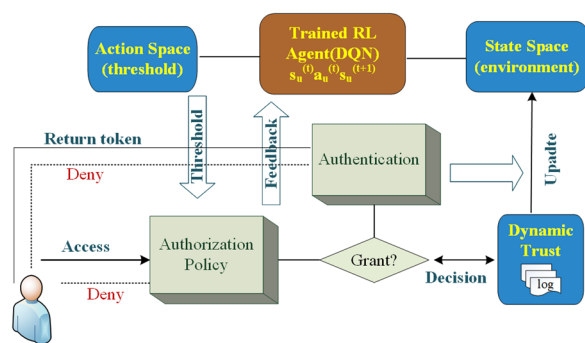


Fig. 4 Work flow of the DR-TBAC system

```

2023-05-16 00:00:00.008 25746 INFO nova.osapi_compute.wsgi.server [req-38101a0b ]
10.11.10.1 "GET /v2/servers/detail HTTP/1.1" status: 200 len: 1893 time: 0.2477829

2023-05-16 00:00:00.272 25746 INFO nova.osapi_compute.wsgi.server [req-9bc36dd9 ]
10.11.10.1 "GET /v2/servers/detail HTTP/1.1" status: 200 len: 1893 time: 0.2577181

2023-05-16 00:00:04.500 2931 INFO nova.compute.manager [req-3ea4052c ]
[instance:b9000564] VM Started (Lifecycle Event)

2023-05-16 00:00:04.562 2931 INFO nova.compute.manager [req-3ea4052c ]
[instance:b9000564] VM Paused (Lifecycle Event)

2023-05-16 00:00:10.279 25743 INFO nova.api.openstack.compute.server_external_events[req-
ab451068 ] Creating event network-vif-plugged:e3871ffd for instance b9000564

2023-05-16 00:00:10.302 2931 INFO nova.virt.libvirt.driver [-]
[instance: b9000564] Instance spawned successfully.

2023-05-16 00:00:10.302 2931 INFO nova.compute.manager[req-d6143bd4 ]
[instance: b9000564] Took 41.81 seconds to spawn the instance on the hypervisor.

```

Fig. 5 OpenStack log entries

Date	Time	Component	Content	Status	W_1	W_2	W_3	Experience tuple
2023/5/16	00:00:00.008	nova.osapi_compute.w	GET /v2/servers/detail	200	0	0.3	0	[(0.5,0.0,3.0), 2, 1, (0.52,0.0,3.0)]
2023/5/16	00:00:00.272	nova.osapi_compute.w	GET /v2/servers/detail	200	0	0.3	0	[(0.52,0.0,3.0), 2, 1, (0.6,0.0,8.0)]
2023/5/16	00:00:04.500	nova.compute.manager	VM Started	ok	0	0.8	0	[(0.6,0.0,8.0), 2, 1, (0.6,0.0,8.0)]
2023/5/16	00:00:04.562	nova.compute.manager	VM Paused	ok	0	0.8	0	[(0.6,0.0,8.0), 2, 1, (0.64,0.0,8.0)]
2023/5/16	00:00:10.279	nova.api.openstack.com	Creating event network-vif-plugged	ok	0	0.8	0	[(0.64,0.0,8.0), 2, 1, (0.6,0.0,8.0)]
2023/5/16	00:00:10.302	nova.virt.libvirt.driver	Instance spawned successfully	ok	0	0.8	0	[(0.6,0.0,8.0), 2, 1, (0.5,1.0,8.1)]
2023/5/16	00:00:10.302	nova.compute.manager	Took 41.81 seconds to spawn the instance on the hypervisor	-	1	0.8	1	[(0.5,1.0,8.1), 8, -1, (*,*,*,*)]

Fig. 6 Examples of experience tuple

files in JSON format that define access controls for its resources to manage and enforce authorization across various OpenStack services. When users attempt to operate within OpenStack, the permission for that operation defined in the policy file is consulted to determine whether the operation is allowed. The following policy rule is a simplified example from *policy.json* file for the Nova service:

```

"admin_or_owner": "is_admin : True or project_id : %(project_id)s"
"compute : start": "rule : admin_or_owner".

```

The rule “*admin_or_owner*” is a common policy rule in OpenStack that allows either the administrator or the owner of the resource to perform the specified action. The rule “*compute : start*” specifies the permissions required to start compute instances, i.e., only administrators or the resource owners can start compute instances within the Nova service.

The original OpenStack policy rules in the DR-TBAC system such as “*compute : start*” are redefined as “*rule : admin_or_owner and $\mu_{tt} \geq 0.75$* ”. OpenStack allows administrators to customize and fine-tune access controls based on their specific security and operational requirements, which provides a flexible way to manage and enforce security policies in the DR-TBAC system.

Offline evaluation

We use 80% normal logs to offline train the trust assessment methodology. The remaining 20% normal logs and abnormal logs are used to evaluate how effectively the

model generalizes to new data and its ability to distinguish between normal and malicious behavior. We use ADAM (Adaptive Moment Estimation) to tune training parameters for optimal training performance in $T = 2000$ epochs. To find the optimal configuration for training the LSTM model, we train the model with different combinations of hyperparameters, including weight decays (wd), mini-batch size (mbs), hidden-layer size (hls), and model-layer number (mln). The average receiver operating characteristic (ROC) curve is shown in Fig. 7, and the optimal configuration of the trust continuous assessment model shows the highest AUC score (0.97) and the lowest EER score (2.04%) with parameters $wd = 10^{-5}$, $mbs = 64$, $mln = 2$, $hls = (25, 25)$.

The RL agent samples experience tuples from the replay buffer to learn rules. To ensure the stability and effectiveness of the learning, we train the DQN with $T = 2000$ epochs, $\gamma = 0.1$, $\tau = 0.1$, $w_t = 0.2$, $w_f = 0.1$, and select the DQN optimal hyperparameters based on predefined evaluation metrics:

- (1) Temporal difference (TD) error represents the discrepancy between the predicted Q-value and the actual observed Q-value (Paszke et al. 2019). We collect the TD errors over the last K epochs and calculate the variance: $\sigma_{(TD\ error)}^2 = \frac{1}{K} \sum_{(T-K)}^K (TD\ error - E[TD\ error])^2$. A low variance of the TD error indicates that the Q-network is converging consistently, and the agent’s policy is learning more reliably toward the optimal policy for an efficient training phase.
- (2) Monte Carlo (MC) utilizes the value of the discounted cumulative rewards as an estimator of the Q-value function, which measures the equality between the expected estimator and the estimator. We measure the MC in an epoch: $MC = \frac{1}{N} \sum_{n=0}^N \gamma^n r^{(n+1)}$. The variance

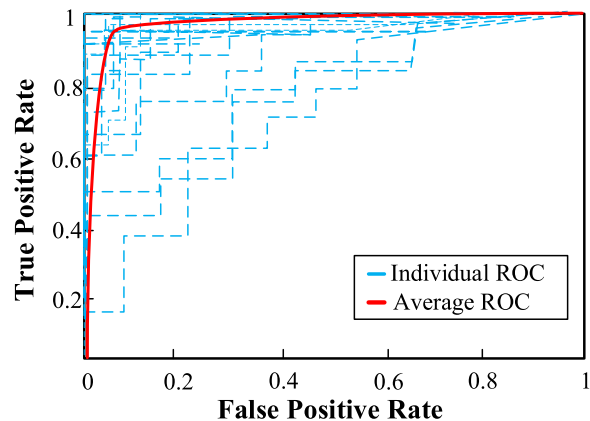


Fig. 7 ROC curve with the optimal parameters

$\sigma_{(MC)}^2 = \frac{1}{K} \sum_{(T-K)}^K (MC - E[MC])^2$ in the last K epochs should be small to improve overall performance.

- (3) Counterfactual policy evaluation (CPE) estimates the performance of a new policy without actually deploying it in the real environment, which is often employed in offline policy evaluation (Foerster et al. 2018). The sequential weighted doubly robust estimator (Jiang et al. 2016) computes the CPE score to reflect the behavior of the new policy, which solves the inefficient and expensive problem of searching for hyperparameters by deploying RL agents online.

In Fig. 8, the optimal configuration of the DQN's hyperparameter metrics shows the lowest TD error and MC and the highest CPE score with hyperparameters $wd = 10^{-5}$, $mbs = 64$, $mln = 2$, $hls = (20, 20)$. The following experiments are analyzed according to the optimal architecture of the DQN and the trust assessment methodology. In Fig. 9, the TD error eventually converges to 0 throughout the training epochs T , which indicates that the DQN estimates the value function accurately and makes better decisions, resulting in a stable and proficient RL agent. The action distribution of permission trust threshold in Fig. 10(a) implies that the RL agent learns different policy π^* to update the rules. The performance of the dynamic rules is further verified by the RL agent launched online. Moreover, we feed the data from the training and validation set separately into the RL agent training process, generating the average action Q-values to validate potential overfitting errors. As shown in Fig. 10(b), the difference between the Q-values for each action is very small, which indicates that the RL agent generalizes well to unlearned data and does not suffer from overfitting.

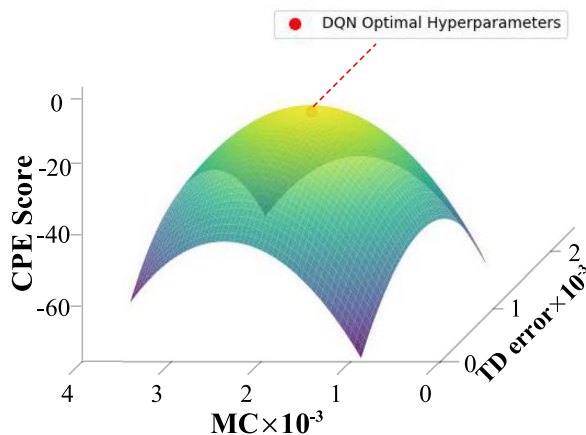


Fig. 8 DQN's hyperparameter

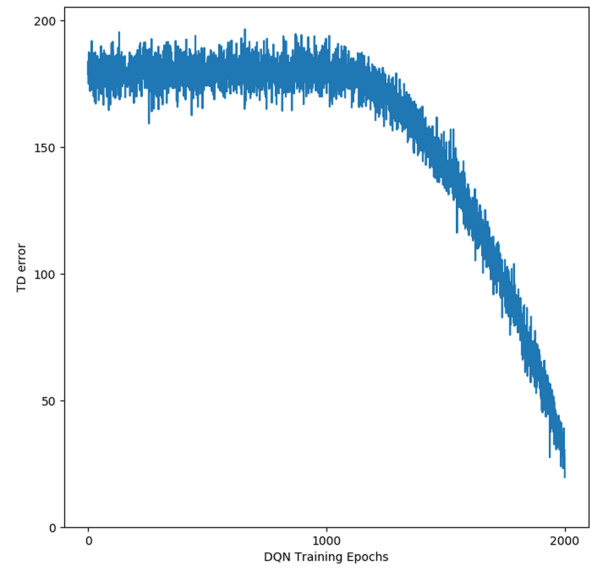


Fig. 9 Stable performance of the RL agent

Online evaluation

We online launch the DR-TBAC system for $T_{online} = 10000$ epochs and consider several representative users' behaviors concerning the constant request action "compute : start" to evaluate the performance changes of dynamic authorization and permission transitions before and after the introduction of dynamic rules.

- (1) user 1 with historical trust 0.8 securely executes all requested permissions.
- (2) user 4 with historical trust 0.1 maliciously executes all requested permissions during training.
- (3) user 12 with historical trust 0.7 executes malicious operations that violate permissions for a period of time but safely executes all requests the rest of the time.
- (4) user 36 with historical trust 0.7 always performs malicious operations outside the requested permissions at a low frequency.

The following experiments analyze the first 1000 epochs of T_{online} . As we can observe from Fig. 11c, static rules

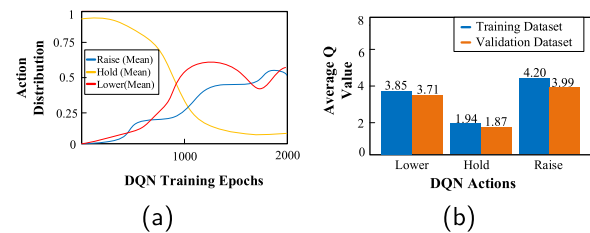


Fig. 10 The RL agent's offline evaluation

and dynamic rules always grants the permissions requested by the user 1 since the continuous increase in user trust [see Fig. 11b] always satisfies the authorization condition. Although the RL agent obtains more positive reward feedback to continuously raise trust thresholds at epochs 900 [see Fig. 11a], the rate of increase of trust thresholds has not been able to keep up with the user's current level of trust. Therefore, users who have been performing well will continue to be trusted by the DR-TBAC system. Figure 11d presents the Q-values for all the user's actions, and Fig. 11a confirms that the DQN selects the action with the maximum Q-value at each training epoch.

In Fig. 12c, since the user 4's historical trust value does not activate the permission of policy “compute : start”, the static rules always deny authorization as the user's trust gradually decreases [see Fig. 12b]. The dynamic rules also consistently deny authorization to the user because the RL agent determines “Hold” or “Raise” action [see Fig. 12a] based on the receipt of negative reward feedback, making it impossible for malicious users to access the OpenStack. In Fig. 12d, the rolling mean of “Hold” and “Raise” is higher than the rolling mean of “Lower”, which is reflected in the fact that the dynamic rules preserve the existing rules or generates new stricter rules for malicious users who are unable to access the system in Fig. 12a.

In Fig. 13b, user 12 obtains the permission “compute : start” in the online launch phase, and the trust value continuously increases since user 12 has been behaving well until epochs 180. However, the trust assessment methodology tracks malicious behaviors during epochs 180-250, and the trust value drops significantly in the aforementioned epochs. In Fig. 13c, the static rules maintain authorization to user 12 after epochs 250 since

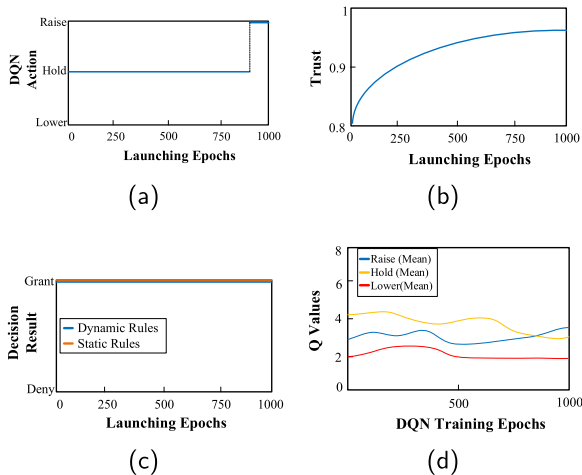


Fig. 11 User 1 authorization process

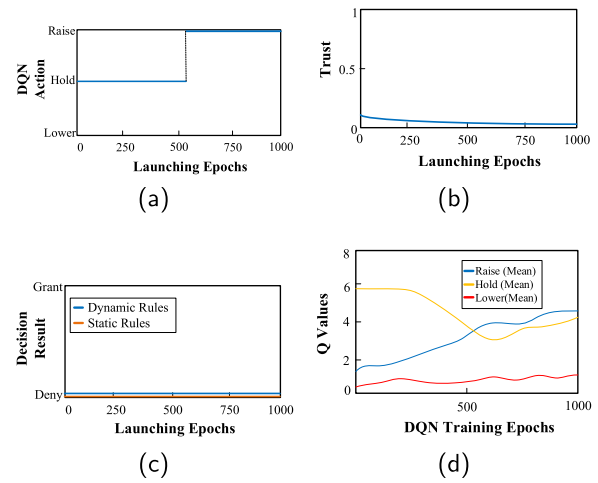


Fig. 12 user 4 authorization process

the current trust value still satisfies the policy rule that is higher than the trust threshold (0.75), and the user may perform more malicious behaviors until the trust value falls below the trust threshold, resulting in a compromised system. On the other hand, the dynamic rules revoke the current permission at epochs 180 since the RL agent raises the trust threshold [see Fig. 13a] according to the dropping user trust and the negative reward value fed back from the system, resulting in the user no longer satisfying the policy conditions. Finally, the DR-TBAC system will consider authorizing the user again until the RL agent lowers the trust threshold based on the user's consistently good performance over time, which minimizes the delayed processing time after an attack is detected and damage to the system. As we can observe in Fig. 13d, the Q-value changes for all actions are consistent with the actions learned by the RL agent.

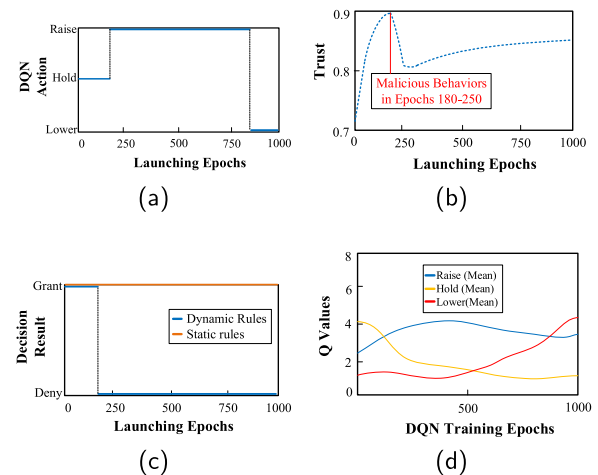


Fig. 13 user 12 authorization process

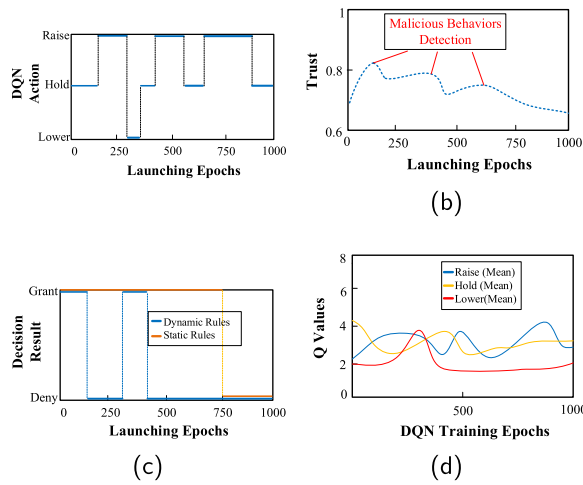


Fig. 14 user 36 authorization process

Table 3 The Positive and Negative Predictive Values

	DR-TBAC			
	NFBV=-500	NFBV=-10	NFBV=-3	NFBV=-1
PPV%	43.66	59.37	89.11	100
NPV%	99.76	99.61	99.35	99.30

In Fig. 14b, the trust assessment methodology tracks malicious behaviors at epochs 120, 360, 650, and the trust value drops significantly in the aforementioned epochs. Finally, the static rules in Fig. 14c revoke authorization to user 36 at epochs 750 since the current trust value is less than the trust threshold (0.75). On the contrary, the dynamic rules revoke the authorization as soon as the user performs malicious behaviors, but the user with enhanced trust will not be granted the privilege right away. In Fig. 14a, the RL agent decides the change in the trust threshold based on the Q-value of all the actions [see Fig. 14d].

The reward function can steer the RL agent towards the desired behaviors during the learning process. The system feedback values have a decisive role in the reward function, in which the negative feedback values that indicate the degree of punishment for malicious behavior are particularly important. Therefore, we count the positive predictive values (PPV) and negative predictive values (NPV) of the DR-TBAC system for different negative feedback values in Table 3. The results show that the more negative values the system feeds back, i.e., the stricter it is about the observed malicious behaviors, the lower the PPV and the higher the NPV. If the system requires very strict rules, the negative feedback value can

be set to -500. If more tolerant rules are required, it is better to set a smaller negative feedback value because the model's overall performance will improve even if it permits some malicious behaviors. Consequently, designing different feedback values or even different reward functions can fulfill the needs of different access control systems.

Seven baseline methods are presented below and analyzed in a comprehensive comparison with the DR-TBAC system.

- (1) The RBAC model that is deployed within the Open-Stack.
- (2) The SVM classifier with RBF kernel (Tay et al. 2020) enables the dynamic and behavior-aware adaptation of authorization policies.
- (3) The Random Forest classifier (Srivastava et al. 2021) determines if access should be given or not according to user trust updated by the Autoencoder-based model.
- (4) The multiclass Adaboost-based model (Zhou et al. 2019) learns user context features for the automatic RBAC role assignment.
- (5) The deep learning model (Yang et al. 2022) dynamically evaluates user trust and authorizes based on short-term tokens.
- (6) The CNN-LSTM model (Kim and Cho 2019) performs classification of users' roles and authorities.
- (7) The MADDPG model (Jin et al. 2021) learns optimal policies combined with user trust profiles generated based on a bidirectional GRU.

Firstly, we define security authorization accuracy as the ratio of the number of correct authorizations added relative to the RBAC model to the total number of correct authorizations. Comparing these seven methods to the static RBAC model during online launch epochs, Fig. 15 illustrates the percentage gain in system security authorization accuracy and reveals the best performance (19% increase) of the DR-TBAC system. The MADDPG-BiGRU method outperforms the other alternatives yielding a 14.78% increase in accuracy because the BiGRU model utilizes user identity as prior knowledge and draws user behavioral traces based on user behavioral logs, and the MADDPG algorithm is fine-tuned to optimize policy scheduling by constructing a state transition prediction model and designing noise simulation function. However, the MADDPG algorithm is suitable for multi-agent and continuous action space scenarios, which increases the computational complexity and reduces the accuracy. The CNN-LSTM model achieves a 12.55% accuracy increase because the CNN is applied to parsed queries to automatically identify

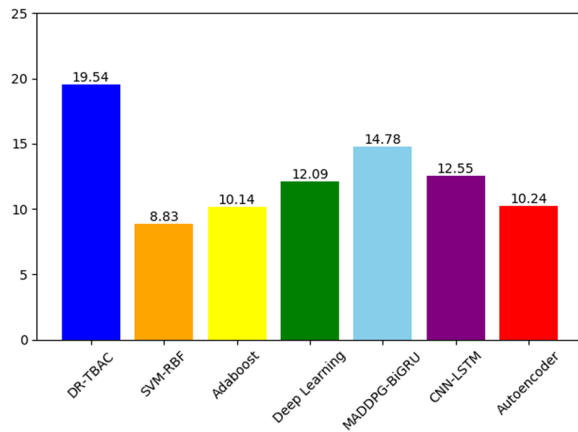


Fig. 15 Comparison of the increase in security authorization accuracy

and extract important features that are utilized for subsequent role classification, and the feature vectors are fed into the LSTM layer to capture the temporal dependencies and patterns within these vectors. The deep learning method increases 12.09% accuracy for leveraging the neural networks with a multi-layer attention mechanism that is used to process complex variable-length sequence data and the SVM that is effective for user classification. The Autoencoder-based model achieves a 10.24% security accuracy increase higher than the Adaboost model without an autoencoding DNN because it optimizes real-time user interactions by learning efficient representations of the input data, feeding the automatically encoded data into the RF classifier and ensemble learning to create decision boundaries and perform final classification. Finally, the SVM model with RBF kernel exhibits the worst result.

Secondly, to compare the performance of the proposed access control schemes, precision, recall, and F1-score are considered evaluation metrics and defined as follows:

$$Precision = \frac{TP}{TP + FP}, \quad (6)$$

$$Recall = \frac{TP}{TP + FN}, \quad (7)$$

$$F1 - score = 2 * \frac{Precision * Recall}{Precision + Recall}. \quad (8)$$

The TP (true positives), TN (true negatives), FP (false positives), and FN (false negatives) statistical results of the DR-TBAC system for the four users are shown in “Appendix: Statistical Results (Tables 5, 6)”. In Table 4, our proposed DR-TBAC system is more accurate and robust compared to the other approaches because the DQN algorithm explores the optimal policy for deriving dynamic rules faster and more accurately based on changes in user behaviors and system decisions to maintain security and accuracy in the face of various challenges and threats.

Conclusion

In this paper, we integrate continuous visibility and flexible rule improvement throughout the entire lifecycle of access control to achieve a more responsive and adaptive security posture. In the proposed DR-TBAC system, we design the trust assessment methodology to generate user trust for each user by processing real-time updated user behavior logs and using user trust as an important factor in the learning process of the RL agent. Secondly, we take user behavior and environment changes into account and apply the DQN algorithm to achieve dynamic adjustment of access control rules for finer-grained privilege adjustment. Thus, the DR-TBAC system is capable of operating as a dynamic policy decision point engine that makes access control decisions for each access request. Extensive experimental evaluations prove the practicability and accuracy of the DR-TBAC system, which strengthens the constraints on user behaviors and the security of the cloud environment.

Future work should attempt to apply other offline deep RL methods and research into how to reduce time overhead and waste of system resources to conduct more rational policy decision points. Alternatively, we can design a rule processing scheme based on the replay experience buffer data format to accommodate different access control policies for different cloud platforms. Finally, the approach will be further extended to implement a policy enforcement point that analyzes and

Table 4 Comparative Evaluation Of The Performance

	DR-TBAC	MADDPG-BiGRU	CNN-LSTM	Deep Learning	Autoencoder	Adaboost	SVM-RBF	RBAC
Precision	0.98	0.94	0.93	0.92	0.89	0.88	0.83	1
Recall	0.97	0.93	0.92	0.91	0.88	0.87	0.83	0.4
F1-score	0.98	0.94	0.92	0.91	0.89	0.88	0.84	0.6

Table 5 Statistics Results For NFBV=-500 and NFBV=-10

	DR-TBAC							
	NFBV=-500				NFBV=-10			
	FP	FN	TP	TN	FP	FN	TP	TN
User 1	0	0	1000	0	0	0	1000	0
User 4	0	0	0	1000	0	0	0	1000
User 12	10	0	100	890	5	1	104	890
User 36	996	0	0	4	980	0	0	20

Table 6 Statistics Results For NFBV=-3 and NFBV=-1

	DR-TBAC							
	NFBV=-3				NFBV=-1			
	FP	FN	TP	TN	FP	FN	TP	TN
User 1	0	0	1000	0	0	0	1000	0
User 4	0	0	0	1000	0	0	0	1000
User 12	4	6	600	390	5	2	600	393
User 36	100	2	685	213	1	2	877	120

forwards traffic through agents to refine the zero-trust architecture.

Appendix: Statistical results

We record the detailed statistical results on the different negative reward feedback values for the four users mentioned in “Online evaluation” section for the first 1000 epochs of T_{online} in the following Tables 5 and 6.

- (1) False positives (FP): The number of normal behaviors prevented.
- (2) False negatives (FN): The number of malicious behaviors permitted.
- (3) True positives (TP): The number of normal behaviors permitted.
- (4) True negatives (TN): The number of malicious behaviors prevented.

Acknowledgements

The authors would like to thank the editors and reviewers for the constructive comments to improve the paper. We would also like to extend our gratitude to all those who have provided assistance and support throughout the development of this paper.

Author Contributions

All authors have contributed to this manuscript and approve of this submission. WR participated in all the work and drafting the article. LC and ZK participated in the baseline experiments and dataset construction. Prof. TBB made a decisive contribution to the content of research and revising the article critically.

Funding

This work is supported by the National Key Research and Development Program of China (Grant No.2020E0HG056105).

Availability of data and materials

Not applicable.

Declarations

Competing interests

The authors declare that they have no Conflict of interest.

Received: 3 January 2024 Accepted: 21 August 2024

Published online: 16 February 2025

References

- Afshar M, Samet S, Usefi H (2021) Incorporating behavior in attribute based access control model using machine learning. In: 2021 IEEE international systems conference (SysCon), pp. 1–8. IEEE
- Alshammari ST, Albeshri A, Alsubhi K (2021) Integrating a high-reliability multicriteria trust evaluation model with task role-based access control for cloud services. *Symmetry* 13(3):492
- Cao Z, Lin C, Zhou M, Huang R (2018) Scheduling semiconductor testing facility by using cuckoo search algorithm with reinforcement learning and surrogate modeling. *IEEE Trans Autom Sci Eng* 16(2):825–837
- Clifton J, Laber E, Laber J (2020) Q-learning: theory and applications. *Annu Rev Stat Appl* 7:279–301
- Cui Z, Xu F, Li Y (2021) Research on security of access control for cloud computing environment. In: security, privacy, and anonymity in computation, communication, and storage: 13th international conference, SpaCCS 2020, Springer, Nanjing, China, Proceedings December 18–20, 2020 13, pp. 145–158
- Damianou N, Dulay N, Lupu E, Sloman M (2001) The ponder policy specification language. In: policies for distributed systems and networks: international workshop, POLICY 2001, Springer, Bristol, UK Proceedings, January 29–31, 2001 pp. 18–38

- Dimitrakos T, Dilshener T, Kravtsov A, La Marra A, Martinelli F, Rizos A, Rosetti A, Saracino A (2020) Trust aware continuous authorization for zero trust in consumer internet of things. In: 2020 IEEE 19th international conference on trust, security and privacy in computing and communications (TrustCom), pp. 1801–1812. IEEE
- Du M, Li F, Zheng G, Srikumar V (2017) Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In: proceedings of the 2017 ACM SIGSAC conference on computer and communications security, pp. 1285–1298
- Foerster J, Farquhar G, Afouras T, Nardelli N, Whiteson S (2018) Counterfactual multi-agent policy gradients. In: proceedings of the AAAI conference on artificial intelligence. 32
- García-Teodoro P, Camacho J, Maciá-Fernández G, Gómez-Hernández J, López-Marín V (2022) A novel zero-trust network access control scheme based on the security profile of devices and users. *Comput Netw* 212:109068
- García-Teodoro P, Camacho J, Maciá-Fernández G, Gómez-Hernández J, López-Marín V (2022) A novel zero-trust network access control scheme based on the security profile of devices and users. *Comput Netw* 212:109068
- Huang W, Xie X, Wang Z, Feng J, Han G, Zhang W (2023) Zt-access: a combining zero trust access control with attribute-based encryption scheme against compromised devices in power iot environments. *Ad Hoc Networks* 145:103161
- Jayaprakash S, Kandasamy K (2018) Database intrusion detection system using octaplet and machine learning. In: 2018 second international conference on inventive communication and computational technologies (ICICCT), pp. 1413–1416. IEEE
- Jiang N, Li L, Li Y (2016) Doubly robust off-policy value evaluation for reinforcement learning. In: international conference on machine learning, pp. 652–661. PMLR
- Jin Q, Wang L, Li Y (2021) Zero-trust based distributed collaborative dynamic access control scheme with deep multi-agent reinforcement learning. *EAI Endorsed Trans Secur Saf* 8(27):e2. <https://doi.org/10.4108/eai.25-6-2021.170246>
- Jing X, Liu Z, Li S, Qiao B, Tan G (2021) A cloud-user behavior assessment based dynamic access control model. *Int J Syst Assur Eng Manag* 8:1966–1975
- Kesarwani A, Khilar PM (2022) Development of trust based access control models using fuzzy logic in cloud computing. *J King Saud Univ-Comput Inform Sci* 34(5):1958–1967
- Khan AJ, Mehruz S, Mehruz A (2023) Fuzzy user access trust model for cloud access control. *Comput Syst Sci Eng*. <https://doi.org/10.32604/csse.2023.023378>
- Kim T-Y, Cho S-B (2019) Cnn-lstm neural networks for anomalous database intrusion detection in rbac-administered model. In: neural information processing: 26th international conference, ICONIP 2019, . Springer, Sydney, NSW, Australia, December 12–15, 2019, Proceedings, Part IV 26, pp. 131–139
- Levine S, Kumar A, Tucker G, Fu J (2020) Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*
- Li T, Jiang Y, Lin C, Obaidat MS, Shen Y, Ma J (2022) Deepag: Attack graph construction and threats prediction with bi-directional deep learning. *IEEE Trans Dependable Secur Computing* 20(1):740–757
- Liu C, Zhu F, Liu Q, Fu Y (2021) Hierarchical reinforcement learning with automatic sub-goal identification. *IEEE/CAA J autom sin* 8(10):1686–1696. <https://doi.org/10.1109/JAS.2021.1004141>
- Mandal S, Khan DA, Jain S (2021) Cloud-based zero trust access control policy: an approach to support work-from-home driven by covid-19 pandemic. *New Gener Comput* 39(3–4):599–622
- Mehraj S, Banday MT, Banday S (2021) A flexible fine-grained dynamic access control approach for cloud computing environment. *Cluster Comput* 24:1413–1434
- Miao Y, Chen C, Pan L, Han Q-L, Zhang J, Xiang Y (2021) Machine learning-based cyber attacks targeting on controlled information: a survey. *ACM Computing Surv (CSUR)* 54(7):1–36. <https://doi.org/10.1145/346517>
- Mnih V, Kavukcuoglu K (2015) Silver: human-level control through deep reinforcement learning. *Nature* 518(7540):529–533
- OASIS: extensible access control markup language (xacml) version 3.0., <http://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>. 2016
- Outchakoucht A, Hamza E-S, Leroy JP (2017) Dynamic access control policy based on blockchain and machine learning for the internet of things. *Int J Adv Comput Sci Appl*. <https://doi.org/10.14569/IJACSA.2017.080757>
- Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, Killeen T, Lin Z, Gimelshein N, Antiga L et al (2019) Pytorch: An imperative style, high-performance deep learning library. *Adv neural inform process syst* 32 DOI:<https://doi.org/10.48550/arXiv.1912.01703>
- Pouyanfar S, Sadiq S, Yan Y, Tian H (2018) A survey on deep learning: algorithms, techniques, and applications. *ACM Computing Surv (CSUR)* 51(5):1–36
- Riad K, Yan Z, Li Y (2017) Multi-factor synthesis decision-making for trust-based access control on cloud. *Int J Coop Inform Syst* 26(04):1750003
- Ribeiro C, Zúquete A, Ferreira P, Guedes PA (2001a) Spl: An access control language for security policies and complex constraints. In: network and distributed system security symposium
- Ribeiro C, Zuquete A, Ferreira P, Guedes P (2001) Spl: an access control language for security policies and complex constraints. In: NDSS 1:89–107
- Srivastava K, Shekhar N (2021) Machine learning based risk-adaptive access control system to identify genuineness of the requester. *Mod Approach Mach Learning Cognitive Sci: A Walkthrough: Latest Trends in AI* 885:129–143
- Stafford V (2020) Zero trust architecture. NIST special publication 800:207. <https://doi.org/10.6028/NIST.SP800-207>
- Sutton RS, Barto AG (2018) *Reinforcement Learning: Introduction*. MIT press
- Tay B, Mourad A, Mourad B (2020) Intelligent performance-aware adaptation of control policies for optimizing banking teller process using machine learning. *IEEE Access* 8:153403–153412
- Uslar M, Specht M, Dänekas C, Trefke J, Rohjans S, González JM, Rosinger C, Bleiker R, Rosinger C (2013) Smart grid security: lec 62351 and other relevant standards. *Standardization in Smart Grids*, 129–146
- Ward R, Beyer B, Beyer R (2014) Beyondcorp: A new approach to enterprise security 39 (6): 6–11
- Yang K, Li D, Zhou L, Cheng K (2022) Research on adaptive dynamic access control model based on blockchain and token. In: *J Phys: conference series*, 216 : 012042
- Yao Q, Wang Q, Zhang X, Fei J (2020) Dynamic access control and authorization system based on zero-trust architecture. In: *Proceedings of the 2020 1st International Conference on Control, Robotics and Intelligent System*, pp. 123–127
- Zhang X (2022) Dynamic access control model based on user access behavior in the internet of things environment. In: 2022 4th international conference on frontiers technology of information and computer (ICFTIC), pp. 1020–1023. IEEE
- Zhao L, Sun M, Yang B, Xie J, Feng J (2022) Zero trust access authorization and control of network boundary based on cloud sea big data fuzzy clustering. *J Intell Fuzzy Syst* 43(3):3189–3201
- Zhao Z, Sun L, Li Y (2021) Attribute-based access control with dynamic trust in a hybrid cloud computing environment. In: proceedings of the 2017 international conference on cryptography, security and privacy, pp. 112–118
- Zhou L, Su C, Li Z, Liu Z, Hancke GP (2019) Automatic fine-grained access control in scada by machine learning. *Future Gener Comput Syst* 93:548–559

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.